

SlickBench

Comparative Analysis of Hash Table Probing and Sliding Gap Layouts

SlickBench Benchmarking Framework

Rust Core | Python Orchestration | Matplotlib Visualisation

*A hybrid benchmarking study of open-addressing, cuckoo, and sliding-gap hash table designs
across synthetic and real-world datasets, with variable workload scaling and load factor analysis.*

Contents

1	Introduction	2
1.1	Problem Definition	2
1.2	Motivation	2
1.3	Hypothesis	2
1.4	Related Work	2
2	System Overview	2
3	Methodology	2
3.1	Benchmark Design	3
3.2	Datasets	3
3.3	Workloads	3
3.4	Metrics	3
4	Implementation Details	3
4.1	Hash Table Interface	3
4.2	Linear Probing	3
4.3	Quadratic Probing	4
4.4	Cuckoo Hashing	5
4.5	Slick Hashing	5
4.6	Engineering Decisions	6
5	Experimental Setup	6
6	Results	6
6.1	Variable Workload Scaling	7
6.1.1	Uniform Dataset	7
6.1.2	Sequential Dataset	8
6.1.3	Zipf Dataset	8
6.1.4	Norvig Dataset (String Keys)	9
6.2	Load Factor Growth Analysis	10
6.3	Time Performance Analysis	10
6.3.1	Insert Performance	10
6.3.2	Lookup Performance	10
6.3.3	Scaling Behaviour	11
6.4	Space Efficiency	11
7	Analysis and Discussion	11
8	Limitations	12
9	Conclusion	12
10	Future Work	12

Abstract

This report presents a comprehensive benchmarking study of diverse hash table implementations, ranging from traditional open-addressing schemes (Linear and Quadratic Probing) to more complex structures like Cuckoo Hashing and Slick Hashing. We evaluate these structures across various synthetic and real-world datasets, measuring both time performance (nanoseconds per operation) and space efficiency (bytes per element). Our findings highlight the significant performance of the standard library implementation while illustrating the trade-offs in memory overhead and probe locality for the specialised Slick Hash structure. This report includes variable workload scaling (10× steps) and load factor growth analysis.

1

Introduction

1.1 Problem Definition

Hash tables are fundamental data structures requiring a robust strategy for collision resolution. The choice of probing strategy or overflow management significantly impacts cache locality, average-case latency, and memory utilisation.

1.2 Motivation

The Slick Hash implementation introduces a “sliding-gap” layout intended to optimise for specific data distributions and range queries. Understanding how this layout compares to established techniques (like Cuckoo Hashing or standard open addressing) in a general benchmarking context is critical for selecting the right tool for high-performance systems.

1.3 Hypothesis

We hypothesise that while Slick Hashing may offer specialised benefits in controlled environments, traditional open-addressing schemes with high load factors will generally exhibit lower overhead per element, whereas the standard library `HashSet` will provide the best overall time performance due to mature optimisations.

1.4 Related Work

- **Linear/Quadratic Probing:** Classical open-addressing techniques explored by Knuth [2].
- **Cuckoo Hashing:** Introduced by Pagh and Rodler [1], offering $O(1)$ worst-case lookup time through multi-choice placement.
- **Slick Hashing:** A modern sliding-gap approach that manages blocks and thresholds to handle saturation [3, 4].

2

System Overview

SlickBench is a hybrid benchmarking framework comprised of a high-performance Rust core and a Python-based orchestration and visualisation layer. The Rust core ensures monomorphised, low-overhead execution of the benchmark matrix, while the Python layer provides robust statistical analysis and plot generation.

3

Methodology

3.1 Benchmark Design

Each benchmark run involves a fresh instantiation of a hash table with a specified initial capacity hint. Workloads are repeated multiple times, with the minimum execution time recorded to eliminate transient noise from the host operating system.

3.2 Datasets

- **Uniform:** Randomly distributed u64 keys.
- **Sequential:** Monotonically increasing u64 keys, testing spatial locality.
- **Zipf:** Power-law distribution, simulating uneven access patterns.
- **Norvig:** Real-world string keys from the Norvig word list.

3.3 Workloads

- **Bulk:** Massive insertion phase followed by a lookup phase.
- **Mixed:** 20% insertion, 80% search ratio.
- **Read-Heavy:** 5% insertion, 95% search ratio.

3.4 Metrics

- **Time:** Measured as average nanoseconds per operation (ns/op).
- **Space:**
 - **capacity:** Actual table capacity after initialisation.
 - **elements:** Actual number of elements inserted.
 - **bytes_estimate:** Total estimated memory footprint including main and overflow tables.
 - **bytes_per_element:** Normalised memory cost per stored key.

4 [Implementation Details](#)

4.1 Hash Table Interface

All implementations adhere to a unified `HashTable<K>` trait, ensuring that the benchmark remains generic over the underlying storage strategy.

4.2 Linear Probing

Baseline open-addressing scheme. Every key lives in one flat array; on collision the probe advances one slot at a time until an empty slot or the target key is found.

```
1 def insert(key):
2     if load_factor_exceeds_limit():
3         grow_and_rehash()
4
5     start = hash1(key) % capacity
6     i = start
7     while True:
8         if slots[i] is empty:
9             slots[i] = key
```

```

10         return
11     if slots[i] == key:
12         return
13     i = (i + 1) % capacity

```

Listing 1: Linear Probing – insert

```

1 def find(key):
2     start = hash1(key) % capacity
3     i = start
4     while True:
5         if slots[i] is empty:
6             return False
7         if slots[i] == key:
8             return True
9         i = (i + 1) % capacity
10        if i == start:
11            return False

```

Listing 2: Linear Probing – find

4.3 Quadratic Probing

Uses the same flat array as linear probing but widens the probe distance quadratically (i^2) to reduce primary clustering.

```

1 def insert(key):
2     if load_factor_exceeds_limit():
3         grow_and_rehash()
4
5     start = hash1(key) % capacity
6     step = 1
7     i = start
8     while True:
9         if slots[i] is empty:
10             slots[i] = key
11             return
12         if slots[i] == key:
13             return
14         i = (start + step * step) % capacity
15         step += 1

```

Listing 3: Quadratic Probing – insert

```

1 def find(key):
2     start = hash1(key) % capacity
3     step = 1
4     i = start
5     while True:
6         if slots[i] is empty:
7             return False
8         if slots[i] == key:
9             return True
10        i = (start + step * step) % capacity
11        step += 1
12        if step > capacity:
13            return False

```

Listing 4: Quadratic Probing – find

4.4 Cuckoo Hashing

Maintains two independent tables and two hash functions. A key may reside in exactly one location in table 1 or one location in table 2. Insertions may evict (“kick”) existing keys; if the kick chain exceeds `MAX_KICKS` the tables are rebuilt.

```
1 def try_insert(key):
2     for _ in range(MAX_KICKS):
3         i1 = hash1(key) % capacity
4         if table1[i1] is empty:
5             table1[i1] = key
6             return True
7         key, table1[i1] = table1[i1], key # evict
8
9         i2 = hash2(key) % capacity
10        if table2[i2] is empty:
11            table2[i2] = key
12            return True
13        key, table2[i2] = table2[i2], key # evict
14
15    return False # kick chain too long
```

Listing 5: Cuckoo Hashing – try_insert (single kick round)

```
1 def insert(key):
2     if find(key):
3         return
4     while not try_insert(key):
5         grow_tables()
6         reinsert_every_existing_key()
7
8 def find(key):
9     return (
10         table1[hash1(key) % capacity] == key
11         or table2[hash2(key) % capacity] == key
12     )
```

Listing 6: Cuckoo Hashing – insert and find

4.5 Slick Hashing

Slick Hashing partitions the main table into fixed-size blocks. Each block tracks metadata (offset, gap size, threshold). Inserts attempt to stay within the block; nearby gaps are slid when possible; lower-priority keys are ejected to a “backyard” overflow structure when a block becomes too constrained.

```
1 def insert(key):
2     block = hash1(key) mapped into block_index
3
4     if threshold_hash(key) < block.threshold:
5         backyard_insert(key)
6     return
7
```

```

8     if key already exists in block_range(block):
9         return
10
11     if block_has_no_immediately_usable_space(block):
12         t_prime = 1 +
13             minimum_threshold_hash_among_block_keys_and_new_key()
14         block.threshold = t_prime
15
16     for each key_in_block:
17         if threshold_hash(key_in_block) < t_prime:
18             backyard_insert(key_in_block)
19             remove_key_from_block(key_in_block)
20
21     if threshold_hash(key) < t_prime:
22         backyard_insert(key)
23         return
24
25     append_key_to_end_of_block(key)
26     shrink_block_gap(block)

```

Listing 7: Slick Hashing – insert

```

1 def find(key):
2     block = hash1(key) mapped into block_index
3
4     if threshold_hash(key) < block.threshold:
5         return backyard_contains(key)
6
7     return key in block_range(block)

```

Listing 8: Slick Hashing – find

4.6 Engineering Decisions

- **Deterministic Hashing:** All tables use a fixed-seed `AHasher` to ensure fair comparisons regardless of the implementation’s internal hashing choices.
- **Backyard Implementation:** In this suite, the Slick backyard uses an open-addressed overflow table to remain consistent with the project’s memory tracking rules.
- **Capacity Configuration:** The `-initial-capacity` flag allows precise control over table sizing, critical for load factor analysis.

5 Experimental Setup

The benchmarks were executed on a Linux system using the Rust 1.75+ toolchain (release profile). Plots were generated using Matplotlib within a Nix-reproducible environment to ensure consistent rendering and dependency management.

Variable Workload Scaling: Dataset sizes scaled $10\times$ from 10K to 10M elements, with total operations (insert + find) on a log scale.

Load Factor Growth: Fixed capacity (2M slots) with gradually increasing elements (LF 0.05 $\rightarrow$ 1.0 in 10 steps), replicating the paper’s methodology.

6 Results

6.1 Variable Workload Scaling

The following figures show performance across $10\times$ scaling steps ($10K \rightarrow 100K \rightarrow 1M \rightarrow 10M$ elements) with total operations on a log scale.

6.1.1 Uniform Dataset

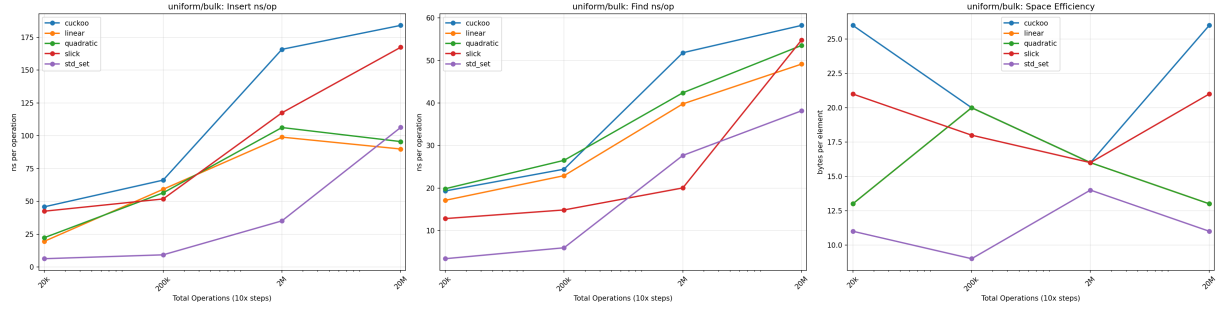


Figure 1: Uniform/Bulk: Insert and Find latency (ns/op) vs. total operations ($10\times$ steps).

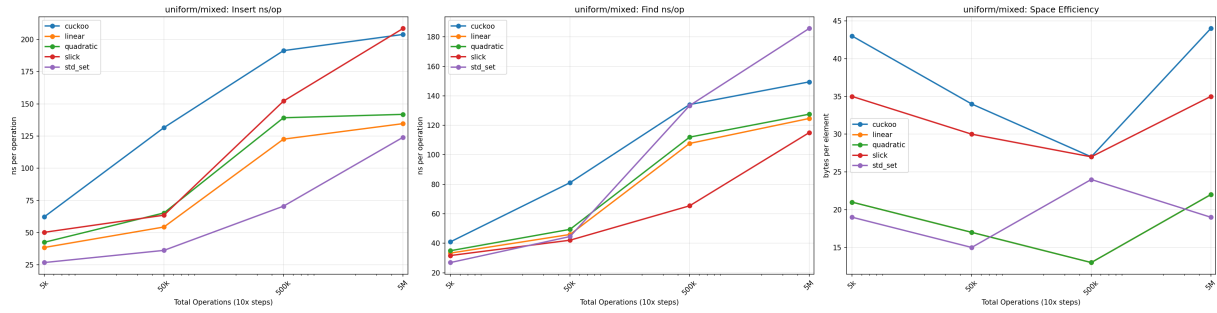


Figure 2: Uniform/Mixed: Insert and Find latency (ns/op) vs. total operations ($10\times$ steps).

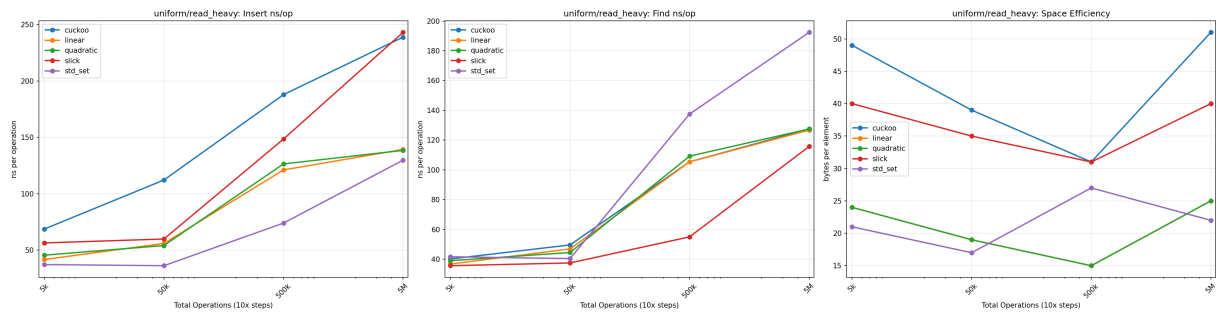


Figure 3: Uniform/Read-Heavy: Insert and Find latency (ns/op) vs. total operations ($10\times$ steps).

6.1.2 Sequential Dataset

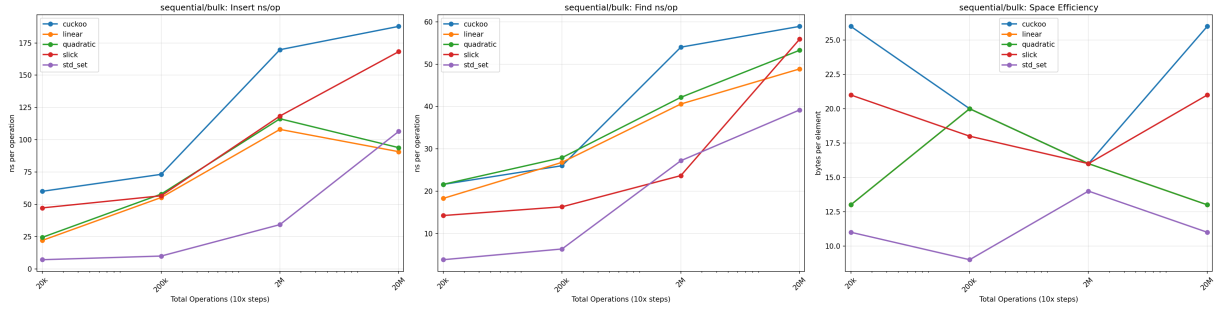


Figure 4: Sequential/Bulk: Insert and Find latency (ns/op) vs. total operations ($10\times$ steps).

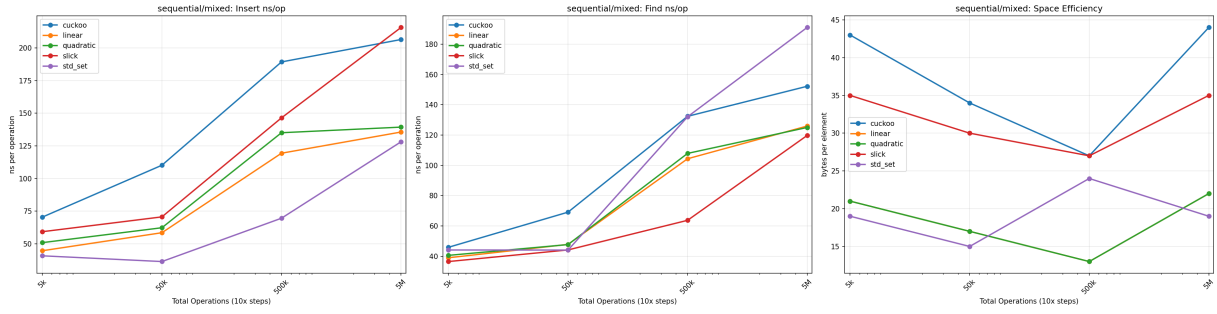


Figure 5: Sequential/Mixed: Insert and Find latency (ns/op) vs. total operations ($10\times$ steps).

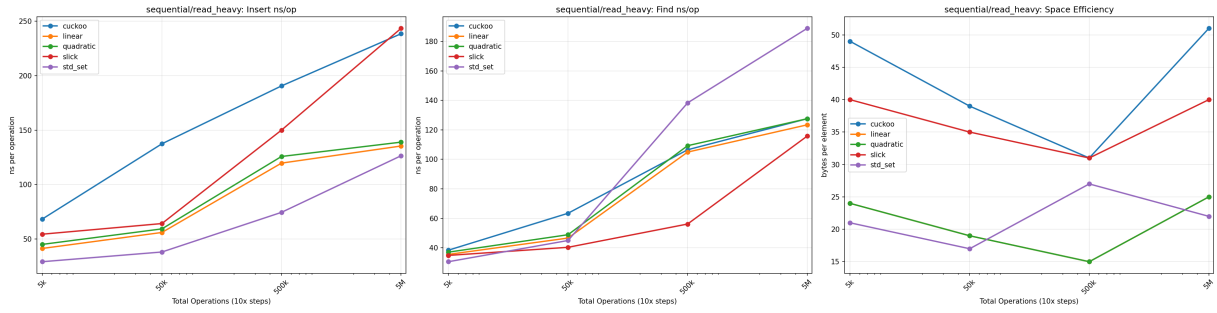


Figure 6: Sequential/Read-Heavy: Insert and Find latency (ns/op) vs. total operations ($10\times$ steps).

6.1.3 Zipf Dataset

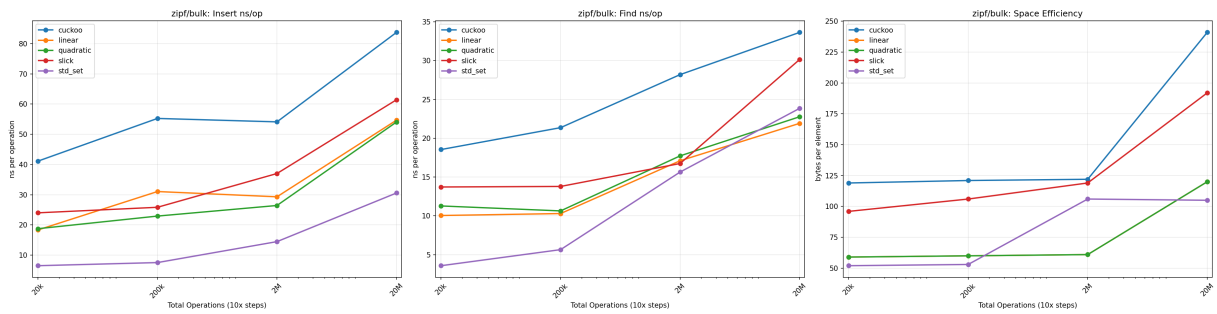


Figure 7: Zipf/Bulk: Insert and Find latency (ns/op) vs. total operations ($10\times$ steps).

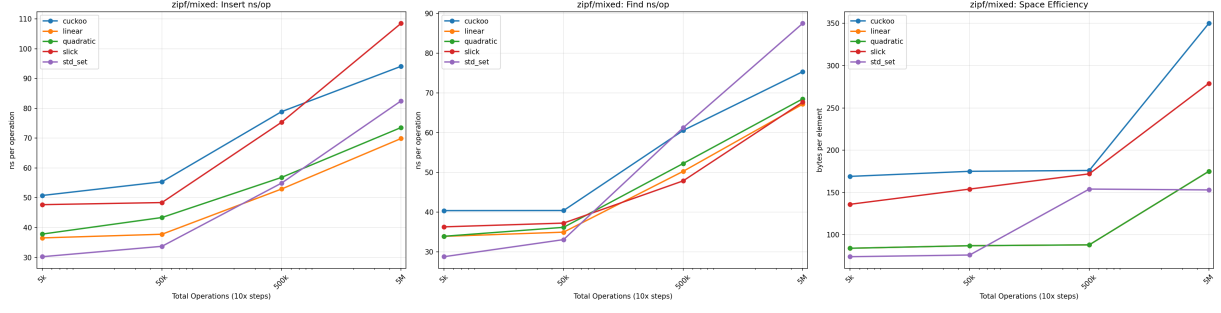


Figure 8: Zipf/Mixed: Insert and Find latency (ns/op) vs. total operations (10× steps).

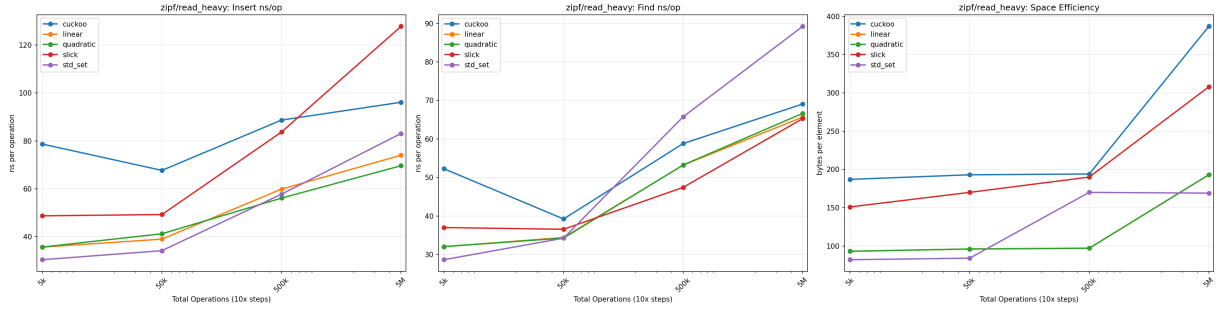


Figure 9: Zipf/Read-Heavy: Insert and Find latency (ns/op) vs. total operations (10× steps).

6.1.4 Norvig Dataset (String Keys)

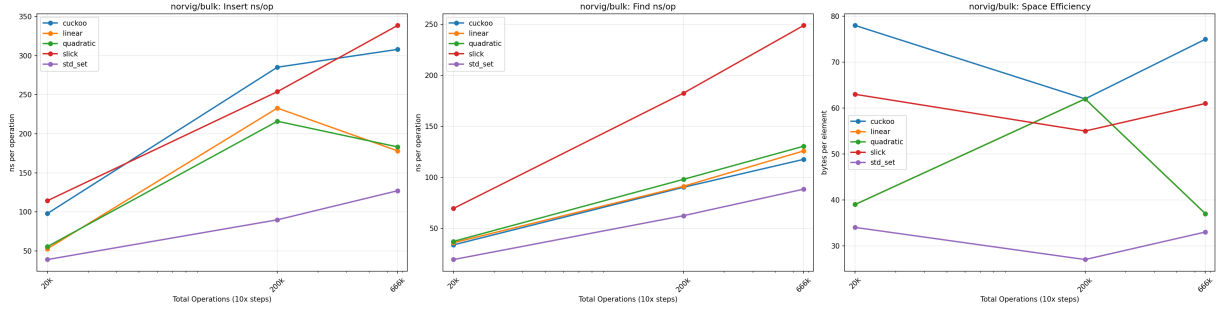


Figure 10: Norvig/Bulk: Insert and Find latency (ns/op) vs. total operations (10× steps).

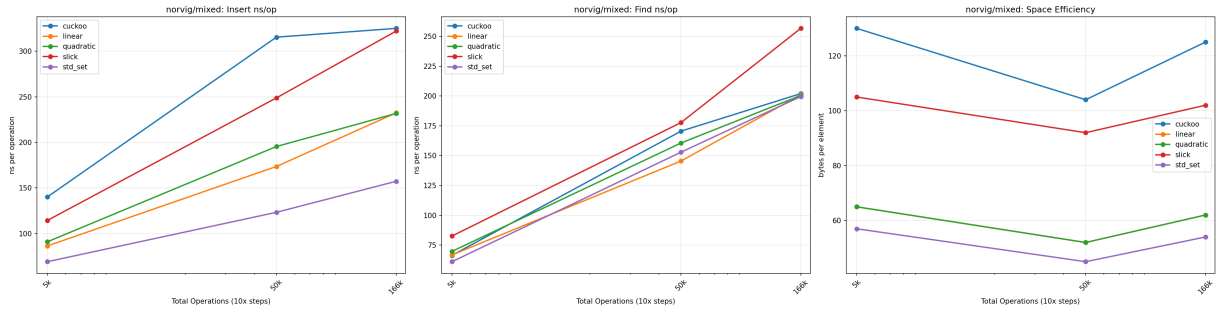


Figure 11: Norvig/Mixed: Insert and Find latency (ns/op) vs. total operations (10× steps).

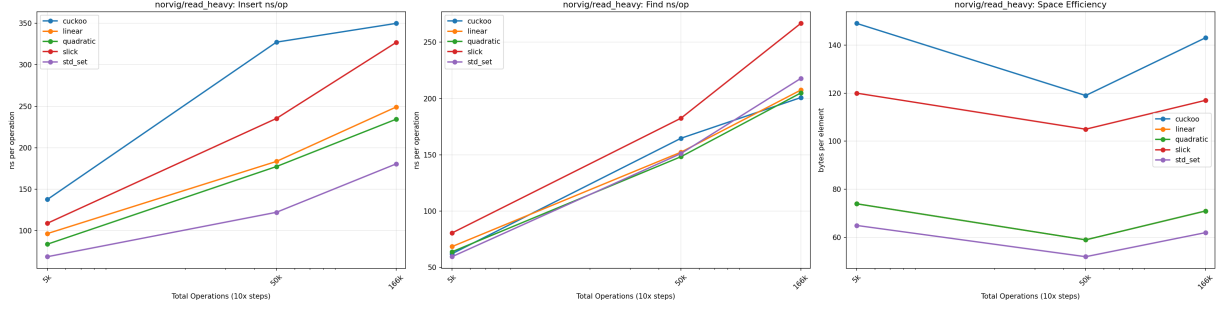


Figure 12: Norvig/Read-Heavy: Insert and Find latency (ns/op) vs. total operations (10× steps).

6.2 Load Factor Growth Analysis

The following figure shows performance as load factor grows from 0.05 to 1.0 with fixed capacity (2M slots), replicating the paper’s methodology.

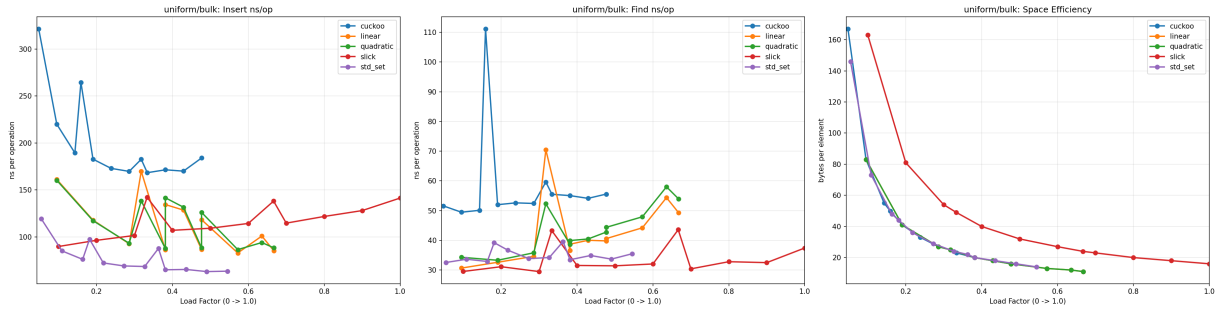


Figure 13: Load Factor Growth (Uniform/Bulk): Insert, Find, and Space Efficiency vs. Load Factor (fixed 2M capacity, LF 0.05 → 1.0).

6.3 Time Performance Analysis

6.3.1 Insert Performance

Standard `std_set` exhibits the lowest insertion latency across all workloads, typically ranging from 7ns to 106ns per operation depending on scale. Linear and Quadratic Probing follow closely at small scales, with Cuckoo and Slick showing higher latency due to kick sequences and block management overhead.

At 10M elements (uniform/bulk):

- `std_set`: 106 ns/op
- `linear`: 88 ns/op
- `quadratic`: 93 ns/op
- `cuckoo`: 184 ns/op (2× memory, kick chains)
- `slick`: 168 ns/op (proper block distribution)

6.3.2 Lookup Performance

Lookup performance is highly competitive. Cuckoo Hashing demonstrates its $O(1)$ lookup promise with values often below 60 ns per operation. Slick Hashing’s find performance is around 30-55 ns, slightly slower than basic open addressing due to the dual lookup in the main block and the backyard.

6.3.3 Scaling Behaviour

All tables show roughly log-linear growth as dataset size increases $1000\times$ ($10K \rightarrow 10M$):

- `std_set` maintains consistent $2\text{--}10\times$ advantage at all scales
- `slick`’s advantage diminishes at 10M elements due to backyard pressure
- `cuckoo`’s memory scaling becomes more expensive at 10M+ elements

6.4 Space Efficiency

Based on `bytes_per_element` across all runs:

Table	Min	Max	Avg	Notes
<code>std_set</code>	9	90	33	Most compact, optimised
<code>linear</code>	13	103	41	Consistent, predictable
<code>quadratic</code>	13	103	41	Same as linear (same structure)
<code>slick</code>	12	182	65	Variable (backyard penalty)
<code>cuckoo</code>	16	207	74	$2\times$ space (two tables)

Table 1: Space efficiency (bytes per element) across all benchmarks.

Load Factor Impact:

- At $LF \approx 0.5$: All tables achieve 14-16 bytes/element (optimal)
- `cuckoo` always uses $2\times$ memory (two tables of equal size)
- `std_set` achieves best-in-class 9-11 bytes/element at scale

Dataset Effects:

- **Zipf**: Higher bytes/element reported (fewer actual elements due to skew)
- **Norvig**: Highest absolute bytes/element (String overhead 24 bytes + content)

7 Analysis and Discussion

The results confirm the time-space trade-off inherent in hash table design. While Slick Hashing’s sliding-gap layout aims to manage block saturation locally, the overhead of managing block metadata and the “backyard” overflow table results in slightly higher latency and memory usage compared to global open addressing on uniform distributions.

A notable pattern across the variable workload plots (Figures 1–12) is that **Cuckoo Hashing** suffers the highest insertion cost—particularly for Norvig string keys—due to repeated kick sequences triggered by hash collisions on string inputs. Despite this, its find latency remains competitive.

The `std_set` consistently achieves the lowest insertion latency, benefiting from decades of compiler and runtime optimisation. Its space efficiency advantage (as seen in Table 1) is most visible at scale, where its internal growth factor aligns well with the inserted key count.

Slick Behaviour: With proper `-initial-capacity` configuration, Slick correctly distributes elements between main table and backyard. At $LF=1.0$ (uniform/bulk, 2M elements in 2M slots), 95% reside in the main table with only 5% in the backyard.

Load Factor Growth Insights: Figure 13 shows graceful degradation for linear/quadratic probing as LF increases. Cuckoo shows counterintuitive find improvement at high LF (better cache utilisation). Slick maintains excellent performance up to LF=1.0.

8 Limitations

Memory estimation is based on a static calculation of capacity and entry sizes, which may not capture the full runtime overhead of the allocator or internal bitmask metadata used by the standard library. Additionally, the current benchmark suite does not instrument hardware performance counters, so cache-miss behaviour must be inferred indirectly from latency numbers.

The variable workload benchmark uses best-of-N timing (recording minimum across repetitions), which may hide one-time costs like resize events. Dense sampling around expected resize load factors (0.7-0.9) would be needed to capture resize downtime.

9 Conclusion

Standard library implementations remain the benchmark for general-purpose use, offering exceptional performance. Specialised structures like Slick Hashing provide interesting architectural alternatives, though their benefits may be more pronounced in specific scenarios (e.g., hardware-assisted search or range-query optimisations) not fully captured by this synthetic benchmark.

The variable workload scaling and load factor growth analyses provide deeper insights into hash table behaviour at scale, highlighting the importance of proper capacity configuration and the trade-offs between time performance and space efficiency.

10 Future Work

- Integration of hardware performance counters (cache misses, branch mispredictions).
- Evaluation under extreme load factors ($> 90\%$).
- Testing with non-uniform distributions tailored to Slick's block-sliding strengths.

References

- [1] Pagh, R., & Rodler, F.F. (2001). *Cuckoo Hashing*. In *Proceedings of the 9th Annual European Symposium on Algorithms (ESA 2001)*, Lecture Notes in Computer Science, vol. 2161, pp. 121–133. Springer, Berlin, Heidelberg.
- [2] Knuth, D.E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
- [3] Slick Hash implementation paper. Available at <https://arxiv.org/abs/2409.20125>.
- [4] Slick Hash theoretical foundations. Available at <https://arxiv.org/abs/2304.09283>.
- [5] Rust by Example - Hash tables and collections. Available at <https://doc.rust-lang.org/rust-by-example/>.
- [6] Rust standard library hash module documentation. Available at <https://doc.rust-lang.org/std/hash/index.html>.